Universidade Cruzeiro do Sul
Ciência da Computação

Aula 13 - Paradigma Orientado a Objetos – Parte 1

Disciplina: Paradigmas de Linguagens de Programação
Professor: Álvaro Colombero

Objetivos



- Revisar a diferença entre abstração de processos e abstração de dados.
- Conceituar tipo de dado abstrato e encapsulamento. Conceituar classes, atributos e métodos.
- Diferenciar a definição de uma classe em C++ e Java. Conhecer a coleta manual de lixo.

Paradigma Orientado a Objetos

- Histórico:
  - Crescente demanda por soluções de software e aumento na complexidade das mesmas (Crise de Software);
  - Necessidade de aumentar a produtividade dos programadores:
    - Reusabilidade.
    - Manutenibilidade.
    - Extensibilidade.
- Os recursos ofertados pelos paradigmas imperativo, funcional e lógico dificultavam estas tarefas.

Paradigma Orientado a Objetos

- **Reusabilidade** é a utilização de conceitos ou soluções previamente elaboradas para a criação de um novo software.
- **Extensibilidade** refere-se às condições associadas à facilidade de se estender o software:
  - Adição de novas funcionalidades.
- **Manutenibilidade** refere-se às condições associadas à facilidade de se alterar o software:
  - Correções de bugs.
  - Alteração de funcionalidades.

Paradigma Orientado a Objetos

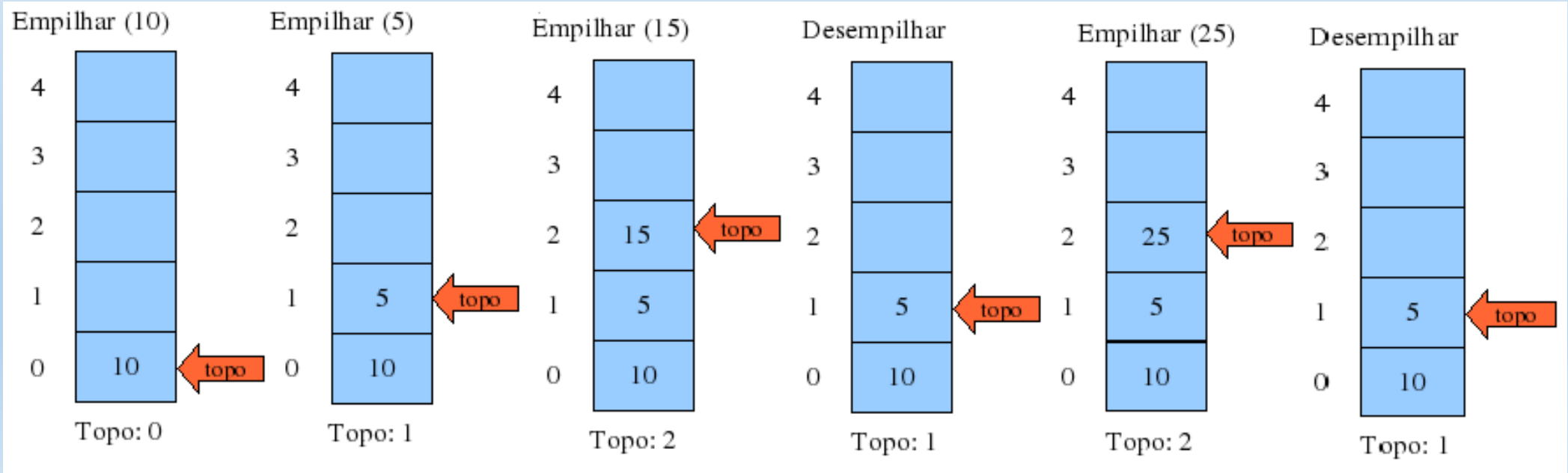
- Histórico:
  - O modelo de abstração por decomposição funcional (abstração de processos) não era adequado para determinados tipos de problemas.
  - Problemas nos pacotes (tipos de dados abstratos dos anos 80) como a inexistência de mecanismos automáticos para inicialização e finalização de valores.
  - Primeiras ideias surgiram no SIMULA 67.
  - Evoluíram até o Smalltalk 80, que é considerada uma linguagem OO pura.

Paradigma Orientado a Objetos

- A Programação Orientada a Objetos está embasada em três recursos principais:
  - **Tipos de dados abstratos (classes).**
  - Herança.
  - Vinculação dinâmica de chamadas à métodos (polimorfismo).

Paradigma Orientado a Objetos

- Exemplo: Implementação de uma Pilha.
  - Pilha é uma estrutura de dados que só permite a inserção e a retirada de elementos no seu topo.



Paradigma Orientado a Objetos

Exemplo 1a: Analise o programa em Java a seguir, que implementa uma pilha usando um vetor e empilha três números lidos do teclado e posteriormente os desempilha e imprime.

Há algum tipo de abstração neste programa?

Paradigma Orientado a Objetos

```java
import java.util.*;

public class Main {
  static int[] pilha = new int[5];
  static int topo = 0;

  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    int elemento;

    for (int i=0;i<5;i++){
      System.out.println("Digite um número: ");
      elemento = Integer.parseInt(input.nextLine());
      if (topo == 5) {
        System.out.println("Erro - pilha cheia");
      }
      else {
        pilha[topo] = elemento;
        topo++;
      }
    }
    for (int i=0;i<5;i++){
      if (topo==0){
        System.out.println("Erro - pilha vazia");
      }
      else {
        topo--;
        elemento = pilha[topo];
        System.out.println(elemento);
      }
    }
  }
}
```

Paradigma Orientado a Objetos

Exemplo 1b: Analise a nova versão do programa e verifique o que mudou.

A única forma de incluir e retirar elementos da pilha é por meio dos métodos push e pop?

O que isto significa em termos de abstração?

Paradigma Orientado a Objetos

```java
import java.util.*;

public class Main {
  static int[] pilha = new int[5];
  static int topo = 0;

  static void push(int n) {
    if (topo == 5) {
      System.out.println("Erro - pilha cheia");
    }
    else {
      pilha[topo] = n;
      topo++;
    }
  }

  static int pop() {
    if (topo==0){
      System.out.println("Erro - pilha vazia");
      return -1;
    }
    else {
      topo--;
      int n = pilha[topo];
      return n;
    }
  }

  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    int elemento;

    for (int i=0;i<5;i++){
      System.out.println("Digite um número: ");
      elemento = Integer.parseInt(input.nextLine());
      push(elemento);
    }

    for (int i=0;i<5;i++){
      elemento = pop();
      if (elemento!=-1){
        System.out.println(elemento);
      }
    }
  }
}
```

Paradigma Orientado a Objetos

Exemplo 1c: Analise a nova versão do programa e verifique o que mudou.

É possível interagir com a estrutura vetPilha no método main?

Se a forma de armazenar os dados na pilha for modificada de vetor (vetPilha) para alocação encadeada, é preciso mudar algo na função main?

O que isto significa em termos de abstração?

```java
class Pilha {
  private int[] vetPilha = new int[5];
  private int topo;

  public void Pilha(){
    topo = 0;
  }

  public void push(int n) {
    if (this.topo == 5) {
      System.out.println("Erro - pilha cheia");
    }
    else {
      this.vetPilha[this.topo] = n;
      this.topo++;
    }
  }

  public int pop() {
    if (this.topo==0){
      System.out.println("Erro - pilha vazia");
      return -1;
    }
    else {
      this.topo--;
      int n = this.vetPilha[this.topo];
      return n;
    }
  }
}
```

```java
public class Main {
  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    int elemento;
    Pilha  pilha = new Pilha();

    for (int i=0;i<5;i++){
      System.out.println("Digite um número: ");
      elemento = Integer.parseInt(input.nextLine());
      pilha.push(elemento);
    }

    for (int i=0;i<5;i++){
      elemento = pilha.pop();
      if (elemento!=-1){
        System.out.println(elemento);
      }
    }
  }
}
```

Paradigma Orientado a Objetos

Exemplo 1d: Analise a nova versão do programa e verifique o que mudou.

A alteração na forma de armazenar os dados na pilha de vetor (vetPilha) para alocação encadeada, fez com que alguma mudança tivesse que ser feita na função main?

O que isto significa em termos de abstração?

Paradigma Orientado a Objetos

```java
import java.util.*;

class Nodo {
  private int valor;
  private Nodo proximo;

  public Nodo(int x, Nodo prox) {
    this.valor = x;
    this.proximo = prox;
  }

  public int getValor(){
    return this.valor;
  }

  public Nodo getProximo(){
    return this.proximo;
  }
}

```

```java
class Pilha {
  private Nodo topo;

  public void Pilha(){
    this.topo = null;
  }

  public void push(int n) {
    Nodo x = new Nodo(n,this.topo);
    topo = x;
  }

  public int pop() {
    if (topo != null){
      int x = topo.getValor();
      topo = topo.getProximo();
      return x;
    }
    else {
      System.out.println("Erro - pilha vazia");
      return -1;
    }
  }
}
```

Paradigma Orientado a Objetos

Main da versão com lista encadeada

```
48 public class Main {
49   public static void main(String[] args) {
50     Scanner input = new Scanner(System.in);
51     int elemento;
52     Pilha  pilha = new Pilha();
53
54     for (int i=0;i<5;i++){
55       System.out.println("Digite um número: ");
56       elemento = Integer.parseInt(input.nextLine());
57       pilha.push(elemento);
58     }
59
60     for (int i=0;i<5;i++){
61       elemento = pilha.pop();
62       if (elemento!=-1){
63         System.out.println(elemento);
64       }
65     }
66   }
```

Main da versão com vetor

```
36 public class Main {
37   public static void main(String[] args) {
38     Scanner input = new Scanner(System.in);
39     int elemento;
40     Pilha  pilha = new Pilha();
41
42     for (int i=0;i<5;i++){
43       System.out.println("Digite um número: ");
44       elemento = Integer.parseInt(input.nextLine());
45       pilha.push(elemento);
46     }
47
48     for (int i=0;i<5;i++){
49       elemento = pilha.pop();
50       if (elemento!=-1){
51         System.out.println(elemento);
52       }
53     }
54   }
```

Paradigma Orientado a Objetos

- Abstração em linguagens de programação:
  - Abstração de processos:
    - Subprogramas
    - Característica básica da programação imperativa.
  - Abstração de dados
    - Tipos de dados abstratos.
      - Atributos (dados)
      - Comportamentos (subprogramas).
    - Base para a programação orientada a objetos.

Paradigma Orientado a Objetos

- “Um tipo abstrato de dados é um invólucro que inclui apenas a representação de dados de um tipo de dados específico e os subprogramas que fornecem as operações para este tipo” (SEBESTA, 2010).
- Resumindo, um tipo de dado abstrato deve fornecer:
  - Uma definição de tipo que permite às unidades de programa declararem variáveis do tipo, mas que oculte a representação de seus dados (encapsulamento).
  - Um conjunto de métodos para manipular os dados contidos no tipo de dado abstrato.

Paradigma Orientado a Objetos

- “Encapsulamento é o mecanismo que permite que constantes logicamente relacionadas, tipos, variáveis, métodos, entre outros, sejam agrupadas em uma nova entidade” (Tucker e Noonan, 2008).
  - Os dados são chamados de atributos ou variáveis.
  - Eles podem aparecer nas classes e nas instâncias.
  - As funções são chamadas de funções membros ou métodos.
  - Todas as instâncias compartilham o mesmo conjunto de métodos, mas cada instância tem seu próprio conjunto de atributos.

Paradigma Orientado a Objetos

- Tipos de dados abstratos são tipos que:
  - Permitem a declaração de variáveis deste tipo.
  - A representação interna das variáveis é ocultada das unidades do programa que as utilizam, sendo as suas operações as únicas formas para acessá-las.
  - Possuem um conjunto de operações para manipular as variáveis criadas.
  - A interface do tipo não depende da representação dos objetos ou da implementação das operações.
  - Permitem que outras unidades do programa, denominadas clientes, utilizem seus serviços.

Paradigma Orientado a Objetos

- O uso de tipos de dados abstratos traz como vantagem a confiabilidade e legibilidade do código, pois:
  - Clientes não podem alterar os valores de seus atributos diretamente, seja de forma intencional ou por acidente.
  - Reduz a quantidade de código e de variáveis que o programador deve estar atento em um trecho do programa.
  - Reduz os conflitos de nomes por gerar escopos menores.

Paradigma Orientado a Objetos

- Implementação de tipos de dados abstratos
  - As primeiras linguagens a incorporarem o conceito de tipo de dado abstrato, implementavam-nos por meio de pacotes.
  - Algumas linguagens como C++, C# e Object Pascal (Delphi) permitem a abstração por meio de classes e registros (structs e records).
  - Java e Smalltalk não possuem o conceito de registros, todas as abstrações devem ser construídas como classes.

Paradigma Orientado a Objetos

O que é uma classe?

Quais são suas principais características?

Paradigma Orientado a Objetos

- Uma classe é uma declaração de tipo que encapsula constantes, variáveis e funções para a manipulação destas variáveis.
  - Ela define estado e comportamento dos objetos.
  - Suas variáveis, chamadas de atributos ou variáveis de instância, podem ser inicializadas pelos construtores e finalizadas pelos destrutores.
  - As demais funções são chamadas de métodos.
  - Define-se como Cliente de uma classe X qualquer outra classe ou método que declara e usa um objeto da classe X.

Paradigma Orientado a Objetos

- Modificadores de acesso: uma classe pode ter atributos e métodos ocultos ou visíveis aos clientes da classe;
  - public – qualquer classe tem acesso.
  - private – apenas a própria classe tem acesso;
  - protected – apenas a própria classe e suas classes derivadas (herança) tem acesso.
- Em geral, os métodos são declarados públicos e os atributos privados ou protegidos, sendo o seu acesso realizado por métodos get e set.

Paradigma Orientado a Objetos

- A criação de uma classe em C++, diferentemente do Java, pode se dar de duas maneiras:
  - Definição completa na própria classe (interface e implementação em único arquivo). Ideal para a programação de sistemas de tempo real.
  - Separação entre a interface e a implementação em arquivos separados (melhora legibilidade e facilita a manutenção, um dos objetivos da programação moderna).

```cpp
//Arquivo pilha.h
class Pilha {
    private:
        int vetPilha[5];
        int topo;
    public:
        Pilha();
        ~Pilha();
        void push(int n);
        int pop();
};
```

```cpp
//Arquivo pilha.cpp
#include <iostream>
#include "pilha.h"

Pilha::Pilha() {
    topo = -1;
}

Pilha::~Pilha(){
}

void Pilha::push(int n) {
    if (topo == tamanho-1) {
        std::cerr << "Erro - pilha cheia\n";
    }
    else {
        topo++;
        vetPilha[topo] = n;
    }
}

int Pilha::pop(){
    if (topo == -1){
        std::cerr << "Erro - Pilha vazia" << std::endl;
        return -1;
    }
    else {
        return vetPilha[topo--];
    }
}
```

Paradigma Orientado a Objetos

- Os objetos dinâmicos de um programa devem ser criados e destruídos:
  - Durante a criação de um objeto, recursos e áreas de memória podem ser alocados.
  - Em algumas linguagens, é dever do programador prever a liberação destes recursos e da memória sempre que necessário durante a destruição do objeto.
  - Algumas linguagens realizam coleta de lixo automática (Java).

Paradigma Orientado a Objetos

```java
import java.util.*;

class Nodo {
  private int valor;
  private Nodo proximo;

  public Nodo(int x, Nodo prox) {
    this.valor = x;
    this.proximo = prox;
  }

  public int getValor(){
    return this.valor;
  }

  public Nodo getProximo(){
    return this.proximo;
  }
}
```

```java
class Pilha {
  private Nodo topo;

  public void Pilha(){
    this.topo = null;
  }

  public void push(int n) {
    Nodo x = new Nodo(n,this.topo);
    topo = x;
  }

  public int pop() {
    if (topo != null){
      int x = topo.getValor();
      topo = topo.getProximo();
      return x;
    }
    else {
      System.out.println("Erro - pilha vazia");
      return -1;
    }
  }
}
```

```java
public class Main {
  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    int elemento;
    Pilha  pilha = new Pilha();

    for (int i=0;i<5;i++){
      System.out.println("Digite um número: ");
      elemento = Integer.parseInt(input.nextLine());
      pilha.push(elemento);
    }

    for (int i=0;i<5;i++){
      elemento = pilha.pop();
      if (elemento!=-1){
        System.out.println(elemento);
      }
    }
  }
```

Paradigma Orientado a Objetos

```cpp
#include <iostream>
#include "pilha.h"

using namespace std;

int main()
{
    Pilha *p1;
    int x;

    p1 = new Pilha();

    for (int i=0;i<3;i++){
        std::cout << "Digite um numero: ";
        std::cin >> x;
        p1->push(x);
    }
    for (int i=0;i<3;i++){
         x = p1->pop();
         if (x!=-1){
           std::cout << x << std::endl;
         }
    }
    delete p1;
    return 0;
}
```

```cpp
#include <iostream>
#include "pilha.h"


Pilha::Pilha() {
    this->topo = NULL;
}

Pilha::~Pilha() {
  while (topo != NULL){
    Nodo* temp = topo->getProximo();
    delete topo;
    topo = temp;
  }
}

void Pilha::push(int n)
{
    Nodo* x = new Nodo(n,this->topo);
    topo = x;
}

int Pilha::pop()
{
    int x = topo->getValor();
    Nodo* temp = topo->getProximo();
```

```cpp
#include "nodo.h"

Nodo::Nodo(int x, Nodo *prox) {
    this->valor = x;
    this->proximo = prox;
}

Nodo::~Nodo() {

}

int Nodo::getValor(){
  return this->valor;
}

Nodo* Nodo::getProximo(){
  return this->proximo;
}
```

Bibliografia

- Material complementar:

TUCKER, A. B.; NOONAN, R. E.; **Linguagens de programação: princípios e paradigmas.** 2. ed. São Paulo: McGrall-Hill, 2009. Capítulo 13.

SEBESTA, R. W.; **Conceitos de linguagens de programação**. 9. ed. Porto Alegre: Bookman, 2011. Capítulos 11 e 12.